Estruturas de Dados e Funções

Estruturas de Dados



Universidade de Brasília
Departamento de Ciência da Computação

O que são Estruturas de Dados?

- Estruturas de dados são formas de organizar, gerenciar e armazenar dados para facilitar o acesso e a modificação.
- Python oferece diversas estruturas de dados básicas:
  - Listas (`list`)
  - Tuplas (`tuple`)
  - Dicionários (`dict`)
  - Conjuntos (`set`)

`list`

- Listas são coleções dinâmicas, ordenadas, e mutáveis de itens.
- Podem conter diferentes tipos de dados.

list

- Listas são coleções dinâmicas, ordenadas, e mutáveis de itens.
- Podem conter diferentes tipos de dados.

```
lista = list()
lista = []
```

`list`

- Dado uma lista:
  ```
  lista = [1, 2, 3]
  ```
- Adicionar elementos:
  - `lista.append(item)`: Adiciona um item ao final.
  - `lista.insert(pos, item)`: Insere um item em uma posição específica.
- Remover elementos:
  - `lista.remove(item)`: Remove a primeira ocorrência de um item.
  - `lista.pop(pos)`: Remove o item em uma posição específica.

`list`

- Fatiamento permite acessar subconjuntos de uma lista.
  - `lista[start:end:step]`
    - `start`: posição inicial (inclusivo)
    - `end`: posição final (exclusivo)
    - `step`: passo.
  - Deixe os índices `start` ou `end` em branco para usar o início ou o fim da lista.

Iterando em list

```
for item in lista:
    print(item)

for i in range(len(lista)):
    print(lista[i])
```

`tuple`

- Tuplas são coleções ordenadas e imutáveis de itens.
- São usadas para dados que não devem ser alterados.

tuple

- Tuplas são coleções ordenadas e imutáveis de itens.
- São usadas para dados que não devem ser alterados.

```
tupla = tuple()
tupla = ()
```

tuple

Como as tuplas são imutáveis, elas não podem ser modificadas depois de criadas. Você pode acessar elementos como em listas:

```
tupla = (1, 2, 3, "Python")
print(tupla[0])
```

Iterando em tuple

```
for item in tupla:
    print(item)

for i in range(len(tupla)):
    print(tupla[i])
```

dict

- Dicionários são coleções desordenadas de pares chave-valor.
- Cada valor está associado a uma chave única.

dict

- Dicionários são coleções desordenadas de pares chave-valor.
- Cada valor está associado a uma chave única.

```
dicionario = dict()
dicionario = {}
```

dict

- Dado um dicionario:

```
dicionario = {"chave1": "valor1", "chave2": "valor2"}
```

- Acessar valores através da chave:

```
dicionario["chave1"]
```

- Adicionar ou modificar pares chave-valor:

```
dicionario["chave3"] = "valor3"
```

- Remover um par chave-valor:

```
dicionario.pop("chave1")
```

Iterando em dict

```
for chave in dicionario:
    print(chave)
```

```
for valor in dicionario.values():
    print(valor)
```

```
for chave, valor in dicionario.items():
    print(chave, valor)
```

`set`

- Conjuntos são coleções não ordenadas e não possuem elementos duplicados.

set

- Conjuntos são coleções não ordenadas e não possuem elementos duplicados.

```
conjunto = set()
```

set

- Adicionar um item ao conjunto:
  `conjunto.add(5)`
- Remover um item:
  `conjunto.remove(1)`
- Operações de conjunto:
  - União: `conjunto1 | conjunto2`
  - Interseção: `conjunto1 & conjunto2`
  - Diferença: `conjunto1 - conjunto2`

Iterando em set

```
for valor in conjunto:
    print(valor)
```

`list` vs `tuple` vs `dict` vs `set`

- Listas: Ordenadas, mutáveis, permitem duplicatas.
- Tuplas: Ordenadas, imutáveis, permitem duplicatas.
- Dicionários: Desordenados, pares chave-valor.
- Conjuntos: Desordenados, sem duplicatas.

Funções

- Funções são blocos de código reutilizáveis que executam uma tarefa específica. As funções podem receber argumentos e podem retornar valores.

Funções

- Funções são blocos de código reutilizáveis que executam uma tarefa específica. As funções podem receber argumentos e podem retornar valores.

```
def saudacao():
    # bloco de código
```

Funções sem parâmetros

- Funções são blocos de código reutilizáveis que executam uma tarefa específica. As funções podem receber argumentos e podem retornar valores.

```
def saudacao():
    print(f"Seja bem vindo!")
```

Funções com parâmetros

- Funções são blocos de código reutilizáveis que executam uma tarefa específica. As funções podem receber argumentos e podem retornar valores.

```
def saudacao(nome, idade):
    print(f"{nome} {idade}")
```

Funções com retorno

- Funções são blocos de código reutilizáveis que executam uma tarefa específica. As funções podem receber argumentos e podem retornar valores.

```
def saudacao(nome, idade):
    return nome + ' ' + str(idade)
```

Funções com parâmetro opcional

- Funções são blocos de código reutilizáveis que executam uma tarefa específica. As funções podem receber argumentos e podem retornar valores.

```
def saudacao(nome='João'):
    return nome + ' ' + 'da Silva'
```

Funções com argumento nomeado

- Funções são blocos de código reutilizáveis que executam uma tarefa específica. As funções podem receber argumentos e podem retornar valores.

```
def saudacao(nome, idade):
    return nome + ' ' + str(idade)

saudacao(idade=19, nome='Maria')
```

Funções com vários argumentos

- Funções são blocos de código reutilizáveis que executam uma tarefa específica. As funções podem receber argumentos e podem retornar valores.

```
def soma(*numeros):
    return sum(numeros)

soma(1, 2, 3, 4)
```

Funções com argumentos nomeados variáveis

- Funções são blocos de código reutilizáveis que executam uma tarefa específica. As funções podem receber argumentos e podem retornar valores.

```
def informacoes(**kwargs):
    for chave, valor in kwargs.items():
        print(f"{chave}: {valor}")

informacoes(nome="Luis", idade=25)
```

Escopo de variáveis

- Variáveis definidas dentro de uma função têm escopo local.
- Variáveis fora das funções têm escopo global.

Escopo de variáveis

- Variáveis definidas dentro de uma função têm escopo local.
- Variáveis fora das funções têm escopo global.

```
def funcao():
    x = 10
    print(x)

x = 5
funcao()
print(x)
```

List Comprehension

- Uma maneira concisa de criar listas em Python.
- Permite gerar listas a partir de iteráveis de forma mais legível e compacta.

List Comprehension

- Uma maneira concisa de criar listas em Python.
- Permite gerar listas a partir de iterável de forma mais legível e compacta.

```
nova_lista = [expressão for item in iterável if condição]
```

```
[x**2 for x in range(10) if x % 2 == 0]
```

Expressões Lambda

- Funções anônimas em Python.
- Útil para criar funções simples de maneira rápida e sem nome.

Expressões Lambda

- Funções anônimas em Python.
- Útil para criar funções simples de maneira rápida e sem nome.

```
lambda argumentos: expressão
```

```
soma = lambda x, y: x + y
print(soma(3, 5))
```

Exceções

- Exceções são erros que ocorrem durante a execução de um programa. Podem interromper o fluxo normal do programa se não forem tratadas.
- Devemos tratar execeções:
  - Para evitar que o programa falhe abruptamente.
  - Para fornecer feedback claro ao usuário.
  - Para permitir que o programa continue em situações controladas.

Tratamento de Exceções

```
try:
    # Código que pode gerar uma exceção
except TipoDeExcecao:
    # Código para tratar a exceção
finally:
    # Código que sempre será executado
```

Tratamento de Exceções

```
try:
    resultado = 10 / 0
except ZeroDivisionError:
    print("Erro: Divisão por zero!")
finally:
    print("Este bloco sempre será executado.")
```

Dúvidas e Perguntas!